UAM

Programación de Aplicaciones de Escritorio

Evaluación 3 – Corte 2

Informe de la Tarea sumativa 1

Reserva concurrente de entradas

Dato	Valor
Modalidad	Trabajo grupal (3 integrantes)
Docente	**[Pendiente]**
Fecha de las pruebas	29 de octubre de 2026
Fecha de entrega	**[Pendiente]**
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2

1. Identificación del caso e integrantes

Caso: Tarea sumativa 1 – Reserva concurrente de entradas (sesión 1, indicadores IL3.1 e IL3.2). **Módulo:** Reserva_Concurrente. **Responsable principal del caso según Git:** André Guido (pendiente de confirmar por el grupo).

Tabla 1. Integrantes del grupo

Integrante	Usuario o autor en Git
Erving Miranda	Erving Miranda
[Pendiente: nombre completo]	**[Pendiente]**
[Pendiente: nombre completo]	**[Pendiente]**

En el historial del repositorio aparecen cuatro autores: Erving Miranda, Andre / André Guido, ValcocerLopez (Victor Mateo Alcocer López) y Argonauta0. La guía indica grupos de tres integrantes, por lo que la lista oficial debe confirmarse antes de la entrega.

2. Objetivo y descripción del funcionamiento

El objetivo es simular la reserva de 20 entradas para un evento atendida por tres agentes que trabajan al mismo tiempo. Cada agente intenta realizar 10 reservas de una entrada, por lo que se presentan 30 solicitudes para 20 entradas: deben confirmarse 20 reservas y rechazarse 10, sin vender nunca más entradas de las disponibles.

La ventana muestra tres tarjetas (Disponibles, Confirmadas y Rechazadas) y una tabla con los intentos, las reservas confirmadas, las rechazadas, el progreso y el estado de cada agente. El botón **Iniciar Simulación** crea los tres agentes. Al terminar, la aplicación verifica el resultado y habilita **Nueva Simulación**, que restablece todos los valores e inicia una ejecución nueva.

3. Hilos y sincronización

3.1 Agentes concurrentes

Cada agente es un objeto AgenteReserva que implementa Runnable. El controlador crea un Thread por agente y llama a start(), de modo que los tres agentes trabajan en segundo plano y la interfaz sigue respondiendo. Antes de cada intento el agente espera una pausa aleatoria de 200 a 500 ms con Thread.sleep(), por lo que el orden entre agentes cambia en cada simulación.

3.2 Sección crítica

El inventario es el recurso compartido. El método reservar() de la clase Inventario está marcado con synchronized, por lo que solo un agente a la vez puede ejecutarlo. La comprobación de disponibilidad y el descuento de la entrada se realizan dentro del mismo método; así, dos agentes no pueden tomar la misma entrada ni dejar el inventario en negativo. La lectura de los tres contadores (obtenerEstado()) también es synchronized para que la interfaz nunca muestre una suma inconsistente.

```
public synchronized boolean reservar() {
    if (disponibles > 0) {
        disponibles--;
        confirmadas++;
        return true;
    }
    rechazadas++;
    return false;
}
```

3.3 Actualización de la interfaz y fin de la simulación

Los agentes no modifican los controles directamente: después de cada intento llaman a actualizarAgente(), que usa Platform.runLater() para que el hilo de JavaFX actualice la tabla y las tarjetas. Un hilo supervisor espera a los tres agentes con join() y luego, también mediante Platform.runLater(), finaliza la simulación. Se usa un hilo aparte porque llamar a join() desde la ventana la congelaría.

3.4 Inicios duplicados

La variable simulacionEnCurso impide crear agentes nuevos mientras hay una simulación activa. Si se pulsa Iniciar durante una ejecución, la aplicación muestra el aviso «Ya hay una simulación en curso: no se crearon agentes nuevos.» y no crea hilos adicionales.

4. Resultados esperados y obtenidos

Tabla 2. Cinco simulaciones completas (esperado: 20 confirmadas, 10 rechazadas y 0 disponibles)

Simulación	Agente 1 (conf./rech.)	Agente 2	Agente 3	Confirmadas	Rechazadas	Disponibles	Resultado
1	6 / 4	6 / 4	8 / 2	20	10	0	Correcto
2	6 / 4	7 / 3	7 / 3	20	10	0	Correcto
3	6 / 4	7 / 3	7 / 3	20	10	0	Correcto
4	7 / 3	6 / 4	7 / 3	20	10	0	Correcto
5	7 / 3	7 / 3	6 / 4	20	10	0	Correcto

En las cinco simulaciones se obtuvo exactamente el resultado esperado y la aplicación mostró el mensaje «Resultado correcto». El reparto entre agentes cambió de una ejecución a otra, lo cual es normal porque la guía no exige un orden fijo. Durante la ejecución se observó que disponibles + confirmadas = 20 (por ejemplo 2 + 18 y 6 + 14) y nunca apareció un inventario negativo.

Tabla 3. Pruebas de aceptación

Prueba	Resultado esperado	Resultado obtenido	Estado
Cinco simulaciones completas	20 confirmadas, 10 rechazadas y 0 disponibles en cada una	Se obtuvo 20 / 10 / 0 en las cinco (Tabla 2)	Cumple
Inventario consistente	Nunca negativo; disponibles + confirmadas = 20	Se cumplió en todas las capturas, incluso durante la ejecución	Cumple
Inicio duplicado	No crear agentes adicionales	Se mostró el aviso y los contadores siguieron con tres agentes	Cumple
Nueva simulación	Restablecer todos los datos antes de iniciar	Cada simulación comenzó desde 20 / 0 / 0 y 0/10 por agente	Cumple
Interfaz operativa	Actualizar desde el hilo de JavaFX sin bloquearse	La ventana respondió durante todas las ejecuciones	Cumple

5. Pruebas realizadas y capturas

Las capturas corresponden a ejecuciones reales de la aplicación realizadas el 29 de octubre de 2026.



Figura 1. *Estado inicial: 20 entradas disponibles y los tres agentes en espera.*



Figura 2. *Segundo clic en Iniciar durante la simulación 2: se muestra el aviso y no se crean agentes nuevos. Disponibles + confirmadas = 20.*



Figura 3. *Simulación 1 terminada: 20 confirmadas, 10 rechazadas y 0 disponibles.*



Figura 4. *Simulación 2 terminada con resultado correcto.*



Figura 5. *Simulación 3 terminada con resultado correcto.*



Figura 6. *Simulación 4 terminada con resultado correcto.*



Figura 7. *Simulación 5 terminada con resultado correcto.*

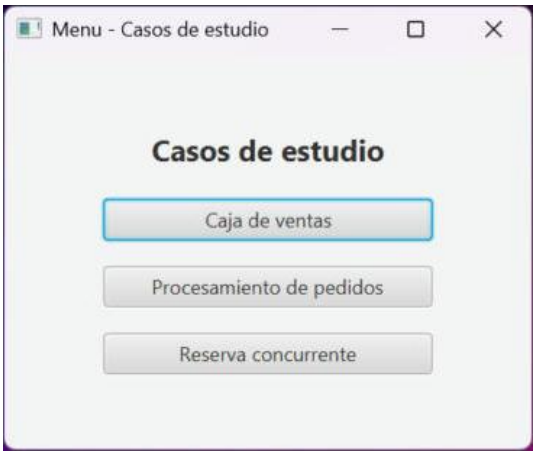


Figura 8. *Menú principal desde el que también se abre este caso (botón Reserva concurrente).*

6. Distribución de responsabilidades

La siguiente tabla resume los aportes que pueden comprobarse en el historial de Git del repositorio. La distribución oficial de responsabilidades (interfaz, lógica concurrente, pruebas y documentación) debe ser confirmada por el grupo.

Tabla 4. Aportes registrados en Git

Autor en Git	Aportes registrados en el repositorio
ValcocerLopez	Estructura inicial del monorepo, menú principal, corrección de los pom.xml y del controlador de la caja; fusión de la solicitud #1 (Reserva_Concurrente).
Andre / André Guido	Caso 1: vista reserva-view.fxml, hojas de estilo, clases Inventario, AgenteReserva y ReservaController; fusión de la solicitud #2 (caja_pedidos).

Argonauta0	Caso 3: vista caja-ventas-view.fxml, CajaVentasController, CajaVentasApplication, modelos CajaCompartida y ProcesadorVentasTask, Launcher y comentarios.
Erving Miranda	Caso 2: PedidosController, ProcesadorPedidos, PedidosApplication, vista pedidos-view.fxml y pruebas JUnit; revisión final del conjunto, README e informes.

Tabla 5. Responsabilidades declaradas por el grupo

Integrante	Responsabilidad en este caso
André Guido	**[Pendiente de confirmar] Según Git: vista, estilos, Inventario, AgenteReserva y ReservaController.**
Erving Miranda	Revisión del conjunto, pruebas manuales, capturas y redacción de este informe.
[Pendiente]	**[Pendiente]**

7. Conclusiones

La aplicación cumple los requisitos del caso: ejecuta tres agentes concurrentes en segundo plano, protege en una sola sección crítica la verificación y el descuento de entradas y mantiene el inventario consistente. Las cinco simulaciones terminaron con 20 reservas confirmadas, 10 rechazadas y 0 entradas disponibles, y el intento de inicio duplicado no creó agentes adicionales. El caso muestra que synchronized basta para evitar la sobreventa cuando la comprobación y la modificación del recurso se realizan juntas.

8. Repositorio y datos de ejecución

Tabla 6. Datos de ejecución

Dato	Valor
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2
Rama revisada	fix/revision-entrega (pendiente de integrar a master mediante revisión del grupo)
Módulo	Reserva_Concurrente (clase de inicio ni.edu.uam.reserva_concurrente.Launcher)
JDK	Microsoft OpenJDK 21.0.13 (JDK 21)
JavaFX	21.0.6 (javafx-controls y javafx-fxml)
IDE	IntelliJ IDEA 2026.2.3
Sistema operativo	Windows 11 Pro
Configuración de IntelliJ	Caso 1 - Reserva concurrente
Forma de ejecución en estas	Directamente desde IntelliJ IDEA con la configuración «Caso 1 - Reserva

pruebas	concurrente».

Para ejecutar desde IntelliJ IDEA: abrir la carpeta raíz del repositorio, seleccionar un SDK 21, elegir la configuración «Caso 1 - Reserva concurrente» o «Menu principal» en la barra superior y pulsar Run. Las configuraciones se incluyen en la carpeta .run/ del repositorio.